Machine Learning Fundamentals

K NEAREST NEIGHBORS

Andy Louwyck
andy.louwyck@vives.be
Academic year 2024 - 2025

hogeschool vives

What?

- K Nearest Neighbors can be applied to
  - **Unsupervised learning** (e.g., recommender systems)
  - **Supervised learning**
    - Regression
    - Classification: this topic

- KNN is a **nonlinear model**

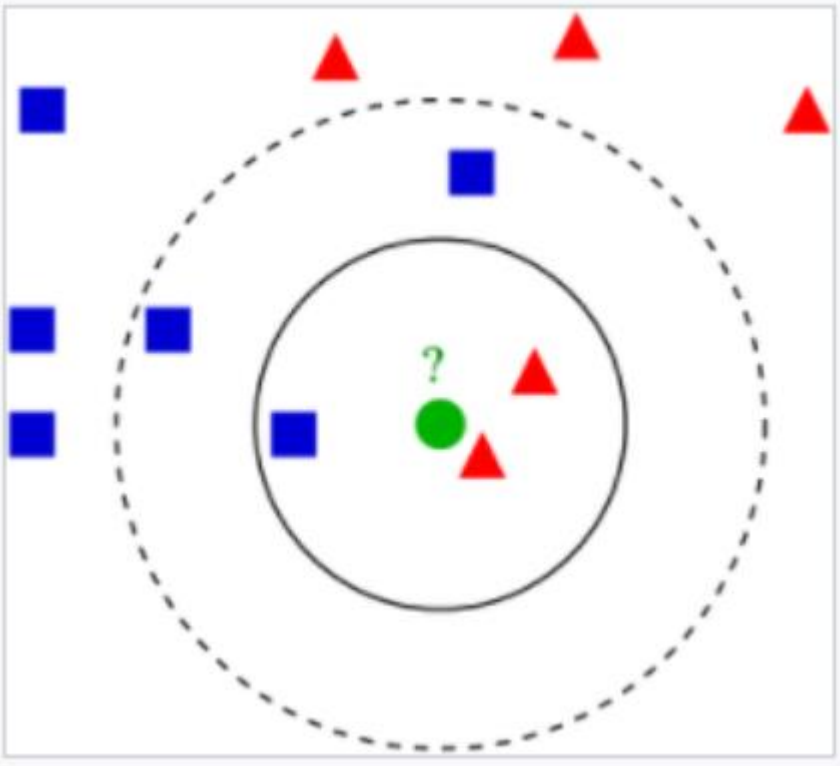
- **Lazy learning** instead of eager learning!

How?

Example:

- 2 classes (binary classification)
  - Red triangles
  - Blue squares
- New point = green circle
  - Which label?
- 3 closest points (K = 3)
  - 2 x red and 1 x blue
  - Label is red
- 5 closest points (K = 5)
  - 2 x red and 3 x blue
  - Label is blue

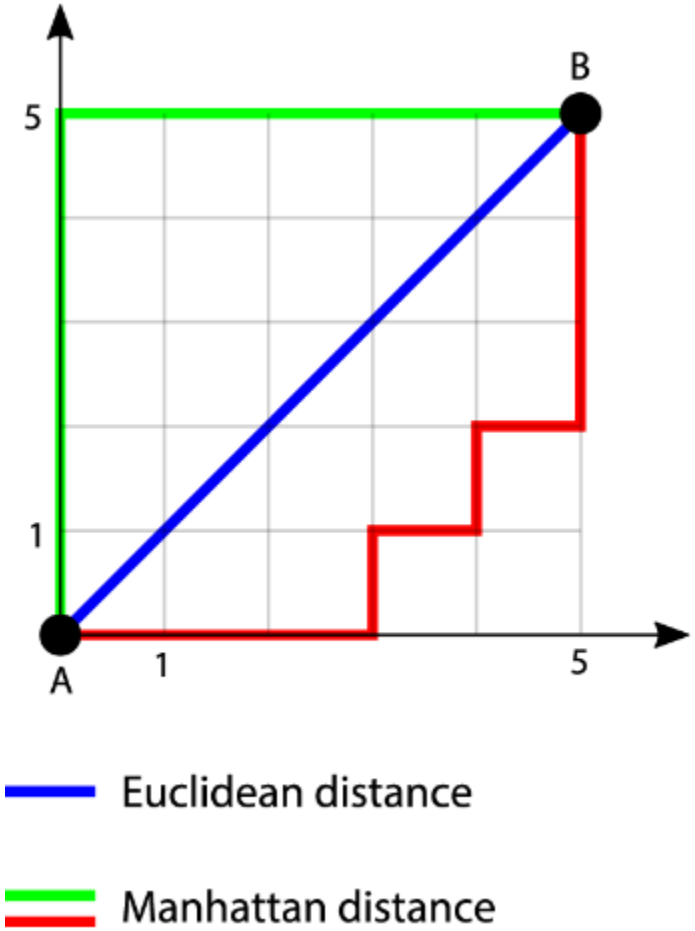
K is a hyperparameter!



Algorithm

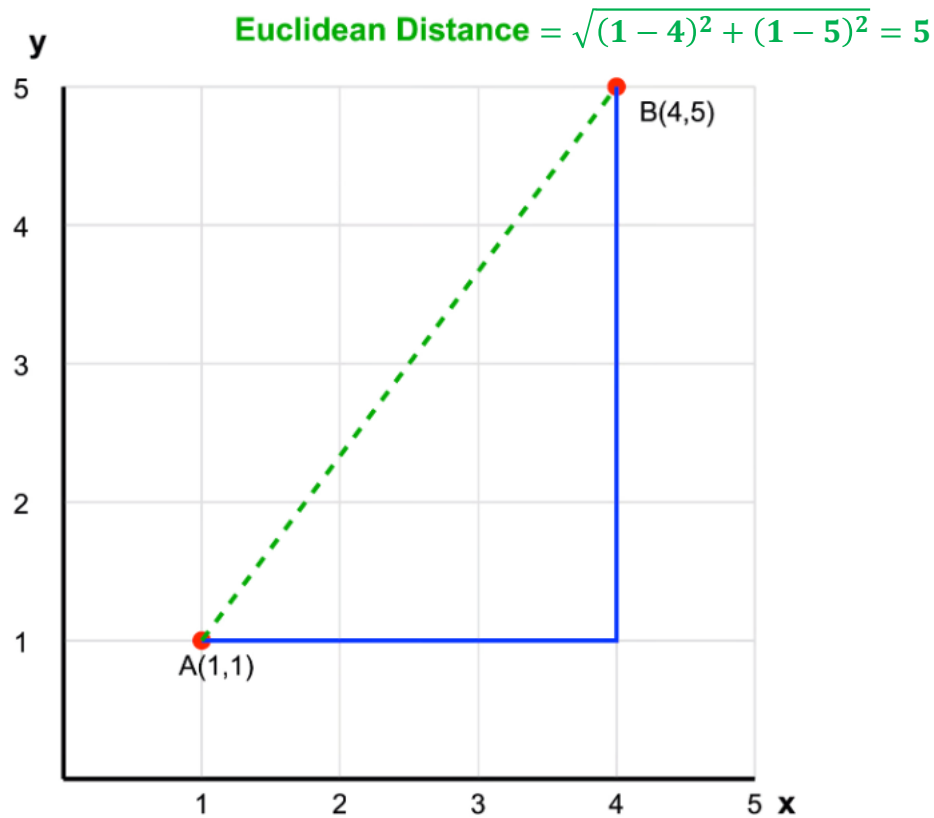
- Determine distance between new data point and given data points
  - Euclidean distance
  - Manhattan distance
  - …

- Sort distances from smallest to largest
  - Taking first K distances
  - Select matching data points
  - Classification:
    - Count labels of the K data points
    - Assign most common label to new data point (= mode)
  - Regression: take average

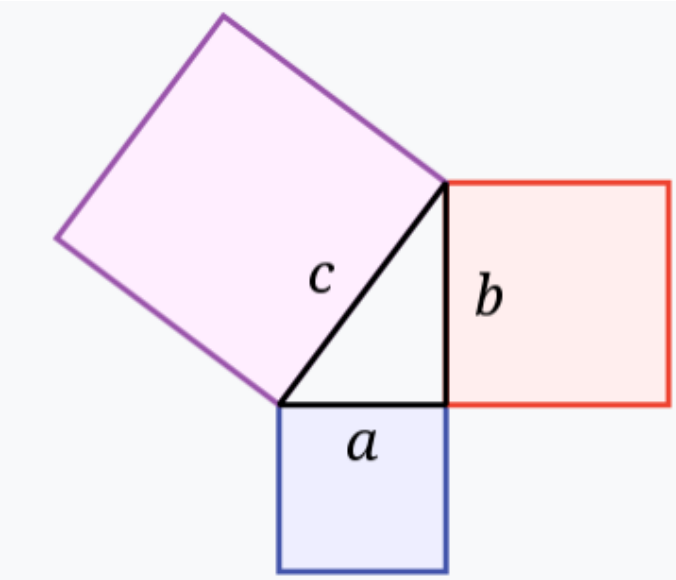


Euclidean distance

Based on Pythagoras' theorem

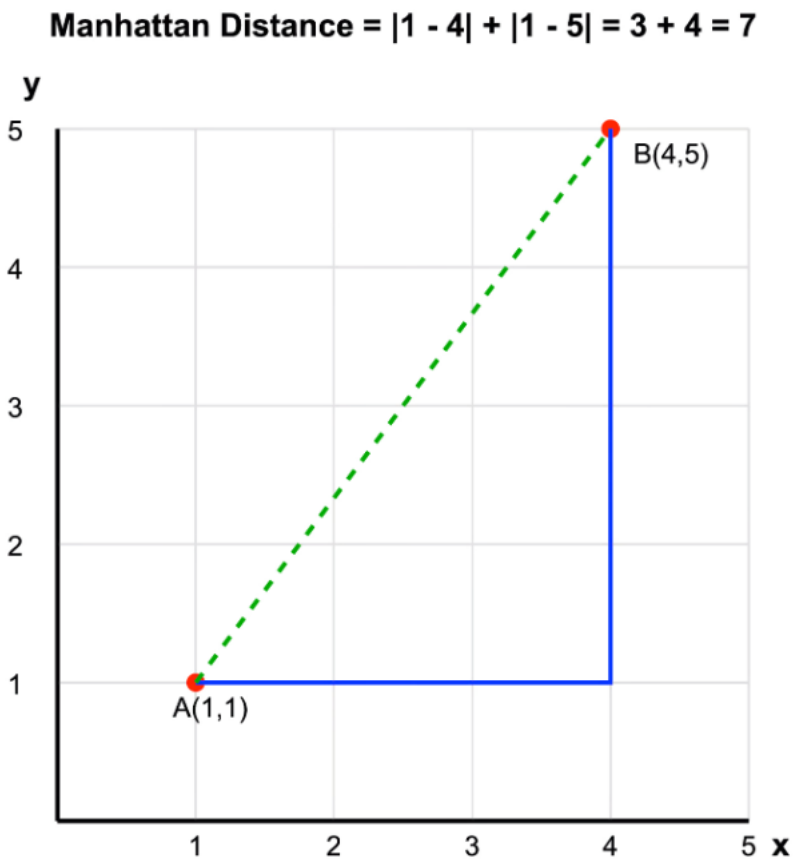


$$a^2 + b^2 = c^2$$



Manhattan distance

= taxicab or city block distance

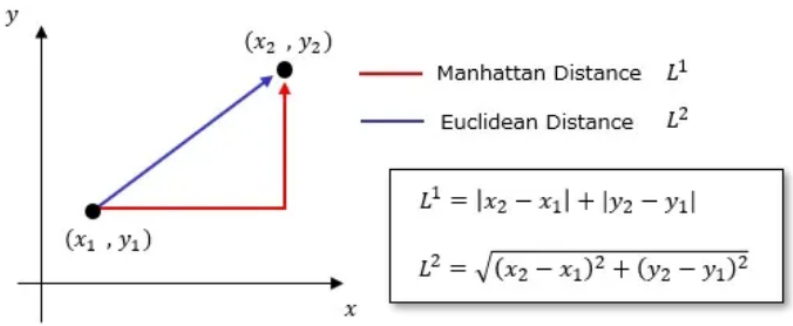


Distances: formulas

- **Euclidean distance** (L2-norm):
  - 1D: $L^2 = \sqrt{(X_2 - X_1)^2}$
  - 2D: $L^2 = \sqrt{\left(X_{1,2} - X_{1,1}\right)^2 + \left(X_{2,2} - X_{2,1}\right)^2}$
  - ND: $L^2 = \sqrt{\sum_j \left(X_{j,2} - X_{j,1}\right)^2}$

- **Manhattan distance** (L1-norm)
  - 1D: $L^1 = |X_2 - X_1|$
  - 2D: $L^1 = \left|X_{1,2} - X_{1,1}\right| + |X_{2,2} - X_{2,1}|$
  - ND: $L^1 = \sum_j |X_{j,2} - X_{j,1}|$

where $X_{j,i}$ is the *j*-th feature of data point *i*



Naive implementation

```
import numpy as np
from scipy.stats import mode

def k_nearest(Xnew, X, y, k=1, distance=None):
    # Xnew is an numpy array with p rows (= examples) and n columns (= features)
    # X and y is the training dataset:
    #   X is an numpy array with m rows (= examples) and n columns (= features)
    #   y is an numpy array with m elements (= labels)
    # k is the number of nearest points (default is 1)
    # distance is a function to calculate distances between points
    #   distance(X, x) is called where x is a new data point with n elements
    #   distance(X, x) returns vector d with m distances
    #   by default Euclidean distance is calculated
    # return value: numpy array yp with p predicted labels for dataset Xnew
    if distance is None:
        distance = lambda X, x: np.sqrt(np.sum(np.square(X - x), axis=1))
    yp = []
    indices = range(X.shape[0])
    for x in Xnew:
        d = distance(X, x)
        sorted_indices = sorted(indices, key=lambda i: d[i])
        k_labels = y[sorted_indices[:k]]
        yp.append(mode(k_labels)[0])
    return np.array(yp)
```

Euclidean distance

sort by distances

mode = label of new point

Scikit-Learn

```
from sklearn.neighbors import KNeighborsClassifier as KNN
knn = KNN(n_neighbors=3)  # n_neighbors = K
knn.fit(X, y)
```

```
KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',
                     metric_params=None, n_jobs=None, n_neighbors=3, p=2,
                     weights='uniform')
```

Hyperparameters:

- `n_neighbors`: number of nearest neighbors K
- `metric`: determines how to calculate distance between data points.
  E.g., Euclidean distance is "`minkowski`" with `p=2`
- `algorithm`: several algorithms implemented
- see documentation

How to determine K?

- Test a range of K values
- Use (cross-)validation to find the optimal value for K
- Make a plot of error vs K
- Use odd values for K to avoid ties in case of (binary) classification
- Consider domain-specific insights to choose K
- Large K values can be computationally expensive

Sources

- Wikipedia:

  https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm

  https://en.wikipedia.org/wiki/Taxicab_geometry

  https://en.wikipedia.org/wiki/Euclidean_distance

  https://en.wikipedia.org/wiki/Pythagorean_theorem

- Online Scikit-Learn documentation:

  https://scikit-learn.org/stable/modules/neighbors.html

hogeschool vives